

Computer Science & Information Systems

Scalable Service – Lab Sheet 3

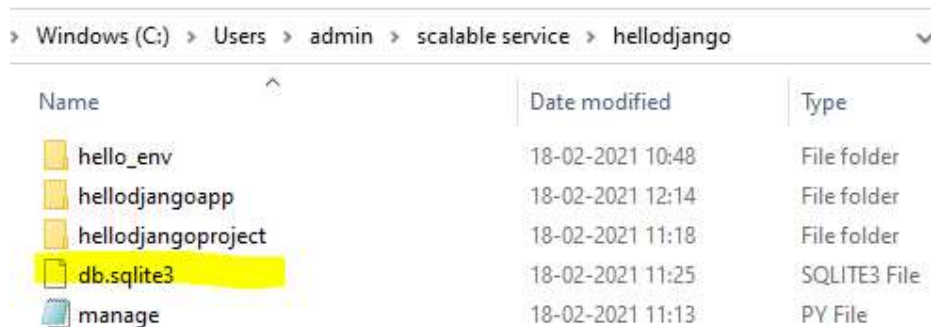
1. Objective:

The main aim of this lab is to allow communication/association between services by using a common database

2. Steps to be performed:

Django App:

By default, when we made our first app and started the server you must have seen a new file in your project directory, named as 'db.sqlite3'. The file is database file where all the data that you will be generating will be stored.



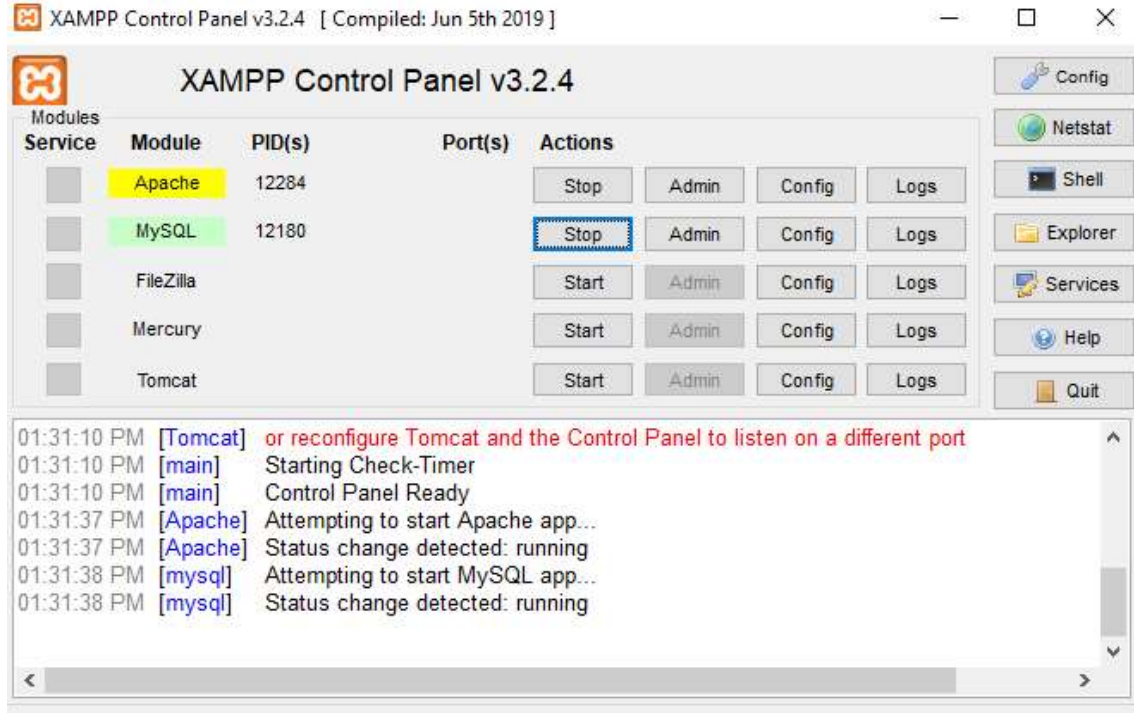
> Windows (C:) > Users > admin > scalable service > hellodjango			
Name	Date modified	Type	
hello_env	18-02-2021 10:48	File folder	
hellodjangoapp	18-02-2021 12:14	File folder	
hellodjangoproject	18-02-2021 11:18	File folder	
db.sqlite3	18-02-2021 11:25	SQLITE3 File	
manage	18-02-2021 11:13	PY File	

In our case, we will be using MySQL and this document will help you to integrate out Django project with MySQL.

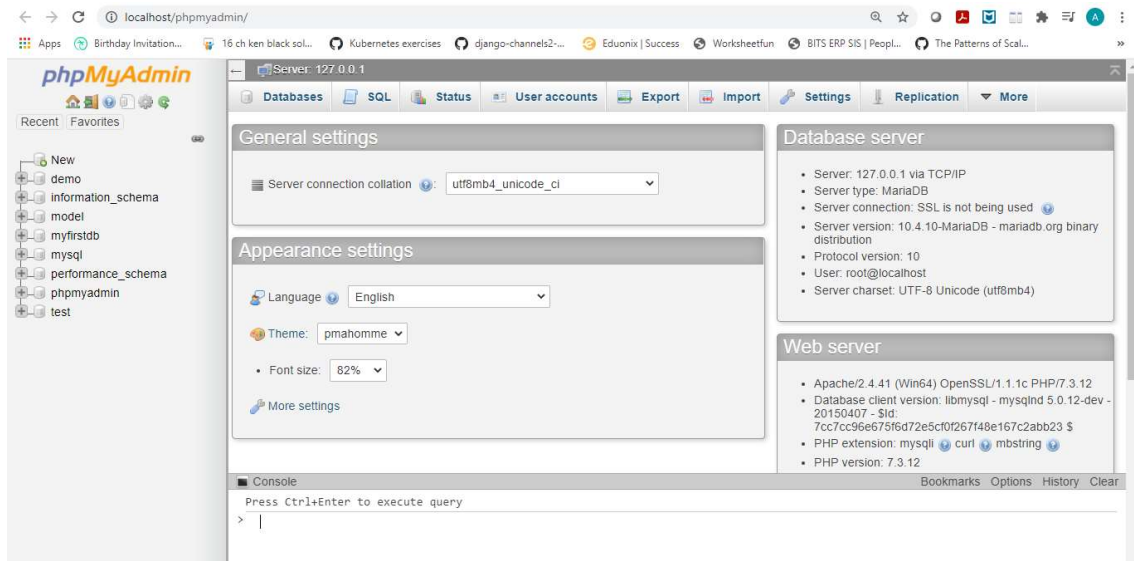
Firstly, Install Xampp which is a free opensource tool which provides you with the Apache server and phpMyAdmin which is the best source for beginner programmers to work with MySQL.

Firstly, open the settings.py file present under the hellodjangoproject and now find the database part in this file

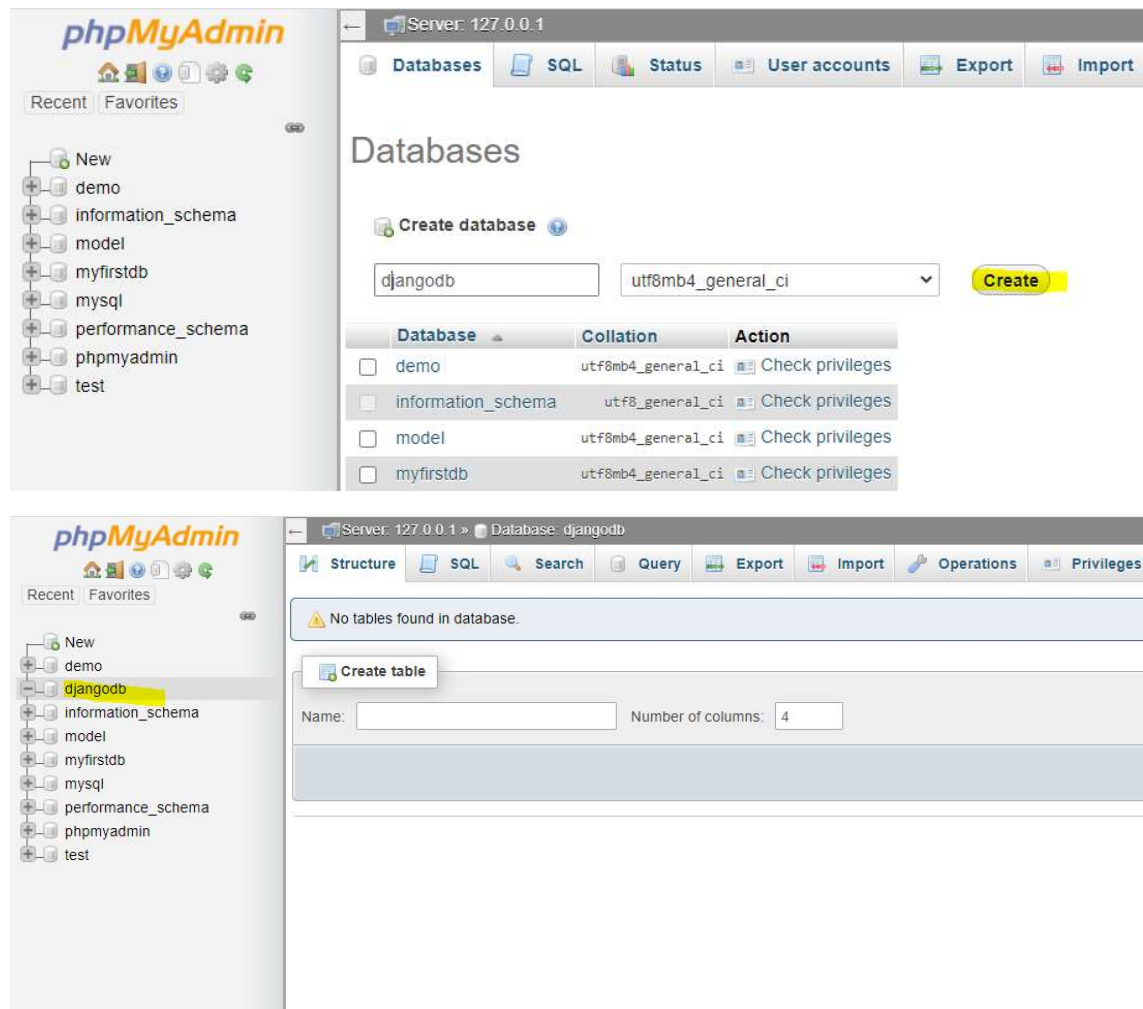
Secondly, you will have to run the Xampp Control Panel and just start 2 services there, Apache and MySQL.



Now, click on the Admin of the MySQL Service, that should open a webpage(offline) looking like this.



On the webpage phpMyAdmin, we will have to create a database for our project. Just Click on the New button as shown here. Then, just fill the desired name of your database and click on create button.



That's it, now we don't need to do anything here. We will be only interacting with python and the models component of Django will prepare everything for us.

In this last step, we will be changing the DATABASE dictionary in our main projects settings.py.

First, install this file via command line:

```
(hello_env) C:\Users\admin\scalable service>cd helلودjango
(hello_env) C:\Users\admin\scalable service\helلودjango>pip install mysqlclient
Collecting mysqlclient
  Using cached https://files.pythonhosted.org/packages/6b/06/a486f3795cf39883815eacec56d00a8fb0e6e46226af5cdfa24ffb3d43e
c/mysqlclient-2.0.3-cp37-cp37m-win_amd64.whl
Installing collected packages: mysqlclient
Successfully installed mysqlclient-2.0.3
```

It will install the Django code for connecting the MySQL Database. After that exchange this code with the DATABASE dictionary in settings.py.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'django',
        'USER': 'root',
        'PASSWORD': '',
        'HOST': '',
        'PORT': '',
        'OPTIONS': {
            'init_command': "SET sql_mode='STRICT_TRANS_TABLES'"
        }
    }
}
```

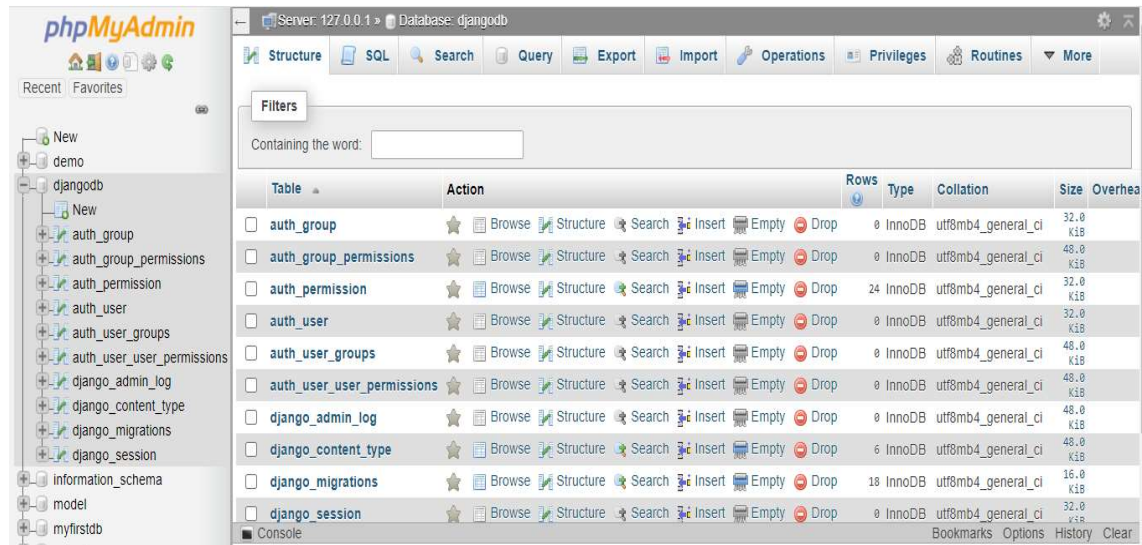
For the last steps just run these two commands to create migrate the changes and run the app:

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 18, 2021 - 13:50:17
Django version 3.1.6, using settings 'hellodjangoproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Now, refresh the phpMyAdmin page and you will get some tables created with just doing this much.



The screenshot shows the phpMyAdmin interface for a database named 'django'. The left sidebar shows the database structure with a tree view. The main area displays a table list for the 'django' database. The table list includes columns for Table, Action, Rows, Type, Collation, Size, and Overhead. The tables listed are: auth_group, auth_group_permissions, auth_permission, auth_user, auth_user_groups, auth_user_user_permissions, django_admin_log, django_content_type, django_migrations, and django_session.

Table	Action	Rows	Type	Collation	Size	Overhead
auth_group	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 Kib	
auth_group_permissions	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 Kib	
auth_permission	✱ Browse Structure Search Insert Empty Drop	24	InnoDB	utf8mb4_general_ci	32.0 Kib	
auth_user	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 Kib	
auth_user_groups	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 Kib	
auth_user_user_permissions	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 Kib	
django_admin_log	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	48.0 Kib	
django_content_type	✱ Browse Structure Search Insert Empty Drop	6	InnoDB	utf8mb4_general_ci	48.0 Kib	
django_migrations	✱ Browse Structure Search Insert Empty Drop	18	InnoDB	utf8mb4_general_ci	16.0 Kib	
django_session	✱ Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_general_ci	32.0 Kib	

Open the models.py file present under hellodjangoapp folder and in that write this code in it

```
from django.db import models

# Create your models here.
class Student(models.Model):
    roll_no = models.TextField()
    name = models.TextField(max_length = 40)
    stud_class = models.TextField()
    department = models.TextField()
```

Now, execute these commands.

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py makemigrations
Migrations for 'hellodjangoapp':
  hellodjangoapp\migrations\0001_initial.py
  - Create model Student
```

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, hellodjangoapp, sessions
Running migrations:
  Applying hellodjangoapp.0001_initial... OK
(hello_env) C:\Users\admin\scalable service\hellodjango>
```

Now open the localhost/phpmyadmin/

Server: 127.0.0.1 » Database: djangodb

Table	Engine	Character Set	Collation	Size
auth_group_permissions	InnoDB	utf8mb4_general_ci		48.0 KiB
auth_permission	InnoDB	utf8mb4_general_ci		32.0 KiB
auth_user	InnoDB	utf8mb4_general_ci		32.0 KiB
auth_user_groups	InnoDB	utf8mb4_general_ci		48.0 KiB
auth_user_user_permissions	InnoDB	utf8mb4_general_ci		48.0 KiB
django_admin_log	InnoDB	utf8mb4_general_ci		48.0 KiB
django_content_type	InnoDB	utf8mb4_general_ci		48.0 KiB
django_migrations	InnoDB	utf8mb4_general_ci		16.0 KiB
django_session	InnoDB	utf8mb4_general_ci		32.0 KiB
hellodjangoapp_student	InnoDB	utf8mb4_general_ci		16.0 KiB
11 tables	Sum			400.0 KiB

Server: 127.0.0.1 » Database: djangodb » Table: hellodjangoapp_student

Table structure

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
2	roll_no	longtext	utf8mb4_general_ci		No				Change Drop More
3	name	longtext	utf8mb4_general_ci		No				Change Drop More
4	stud_class	longtext	utf8mb4_general_ci		No				Change Drop More
5	department	longtext	utf8mb4_general_ci		No				Change Drop More

☐ Check all With selected: ☐ Browse ☐ Change ☐ Drop ☐ Primary ☐ Unique ☐ Index ☐ Fulltext
☐ Add to central columns ☐ Remove from central columns

To View Data stored in database on a page:

Insert some data in the students table that we created earlier

Server: 127.0.0.1 » Database: djangodb » Table: hellodjangoapp_student

Showing rows 0 - 2 (3 total, Query took 0.0014 seconds.)

`SELECT * FROM `hellodjangoapp_student``

☐ Profiling [Edit inline] [Edit] [Explain SQL] [Create PHP code] [Refresh]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

+ Options

	id	roll_no	name	stud_class	department
☐ Edit ☐ Copy ☐ Delete	1	2019cg123001	Advait	Software Systems	CSE
☐ Edit ☐ Copy ☐ Delete	2	2019cg123002	Asmi	Electrical Systems	ECE
☐ Edit ☐ Copy ☐ Delete	3	2019cg123003	Anay	Database Systems	IT

☐ Check all With selected: ☐ Edit ☐ Copy ☐ Delete ☐ Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Now in the student directory, open the views.py file and make following changes in the code

```
from django.http import HttpResponse
from hellodjangoapp.models import Student

def homePageView(request):
    return HttpResponse('Hello, Django!')

def numbers_show(request):
    x = []
    for i in range(10):
        x.append(i)
    return HttpResponse("<h1>List of numbers</h1>The Digits  
are {0}".format(x))

def student_show(request):
    students = Student.objects.all()
    return render(request, "show.html", {'student': students})
```

Now, create a new directory 'templates' in 'hellodjangoapp' folder and an HTML file 'show.html' and put the following code –

```
<!DOCTYPE html>
<html lang="en">
<head>
    <title>CRUD Operations</title>
    <meta charset="utf-8">
    <link rel="stylesheet"
href="https://maxcdn.bootstrapcdn.com/bootstrap/3.4.1/css/bootstrap.min.css">
</head>
<body>
<div class="container">
<table class="table table-striped">
    <thead>
        <tr>
            <th>Student ID</th>
            <th>Roll No</th>
            <th>Student Name</th>
            <th>Class</th>
            <th>Department</th>
        </tr>
    </thead>
    <tbody>
        {% for stud in student %}
        <tr>
            <td>{{stud.id}}</td>
            <td>{{stud.roll_no}}</td>
            <td>{{stud.name}}</td>
            <td>{{stud.stud_class}}</td>
            <td>{{stud.department}}</td>
        </tr>
        {% endfor %}
```

```
</tbody>
</table>
</div>
</body>
</html>
```

Do the following changes to url.py under hellodjangoapp folder

```
#hellodjangoapp/urls.py
from django.urls import path
from .views import homePageView
from .views import numbers_show
from .views import student_show

urlpatterns = [
    path('', homePageView, name='home'),
    path('list/', numbers_show, name='numbers_show'),
    path('studentlist/', student_show, name='student_show')
]
```

Do the following changes to url.py under hellodjangoproject folder

```
#djangohelloproject/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('hellodjangoapp.urls')),
    path('list/', include('hellodjangoapp.urls')),
    path('studentlist/', include('hellodjangoapp.urls')),
]
```

Connecting to the same database with Flask Project

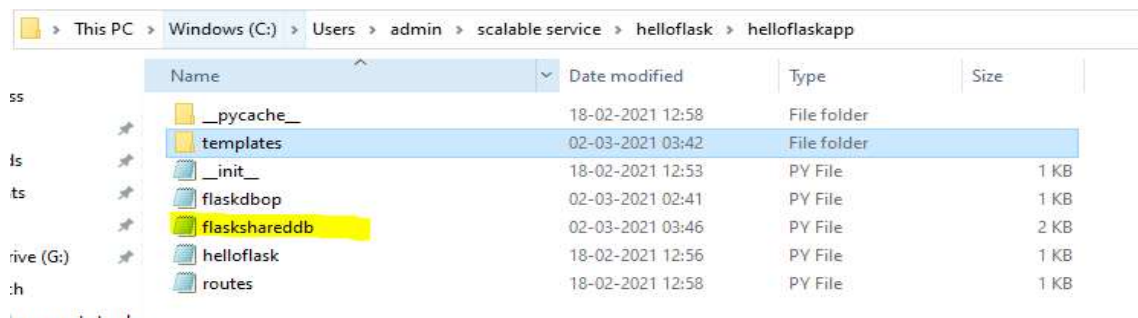
Install Flask-MySQL library in our system

In the same environment (hello_env), run the following command to install the package:

```
(hello_env) C:\Users\admin\scalable service\helloflask\helloflaskapp>pip install flask_mysqldb
Collecting flask_mysqldb
  Downloading https://files.pythonhosted.org/packages/62/e3/955c845efe1aacf30c8e2607470544ede36aa1044f6382c809f13e014104/Flask-MySQLdb-0.2.0.tar.gz
Requirement already satisfied: Flask>=0.10 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from flask_mysqldb) (1.1.2)
Requirement already satisfied: mysqlclient in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from flask_mysqldb) (2.0.3)
Requirement already satisfied: Werkzeug>=0.15 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from Flask>=0.10->flask_mysqldb) (1.0.1)
Requirement already satisfied: itsdangerous>=0.24 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from Flask>=0.10->flask_mysqldb) (1.1.0)
Requirement already satisfied: Jinja2>=2.10.1 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from Flask>=0.10->flask_mysqldb) (2.11.3)
Requirement already satisfied: click>=5.1 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from Flask>=0.10->flask_mysqldb) (7.1.2)
Requirement already satisfied: MarkupSafe>=0.23 in c:\users\admin\scalable service\hellodjango\hello_env\lib\site-packages (from Jinja2>=2.10.1->Flask>=0.10->flask_mysqldb) (1.1.1)
Installing collected packages: flask-mysqldb
  Running setup.py install for flask-mysqldb ... done
Successfully installed flask-mysqldb-0.2.0
```

Using MySQL in Flask application

Under the helloflaskapp folder, create a file named flaskshreddb.py and put the following code



```
from flask import Flask,render_template, request
from flask_mysqldb import MySQL
```

```
app = Flask(__name__)
```

```
app.config['MYSQL_HOST'] = 'localhost'
app.config['MYSQL_USER'] = 'root'
app.config['MYSQL_PASSWORD'] = ""
app.config['MYSQL_DB'] = 'djangodb'
```

```
mysql = MySQL(app)
```

```
@app.route('/shreddbform')
def shreddbform():
    return render_template('shreddbform.html')
```

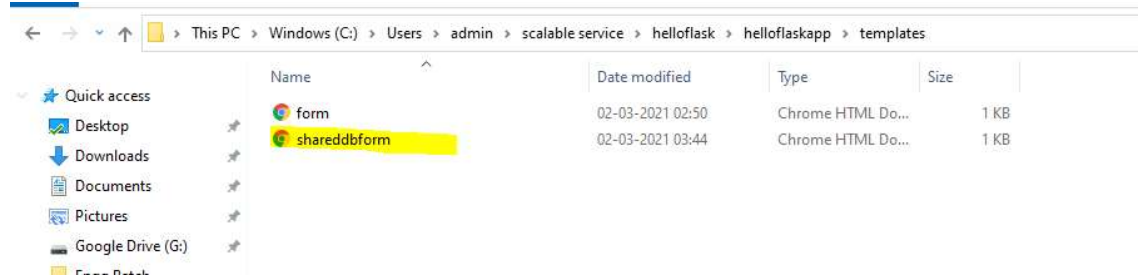
```
@app.route('/login', methods = ['POST', 'GET'])
def login():
    if request.method == 'GET':
        return "Login via the login Form"
```

```
    if request.method == 'POST':
```

```
id = request.form['id']
roll_no = request.form['roll_no']
name = request.form['name']
stud_class = request.form['stud_class']
department = request.form['department']
cursor = mysql.connection.cursor()
cursor.execute(""" INSERT INTO hellodjangoapp_student VALUES(%s,%s,%s,%s,%s,%s)""",(id, roll_no,
name, stud_class, department))
mysql.connection.commit()
cursor.close()
return f"Done!!"
```

```
app.run(host='localhost', port=5000)
```

Create a folder named “templates” under “helloflaskapp”. Under this folder create a file “sharedbform.html” and the following code



```
<form action="/login" method = "POST">
<p>ID <input type = "text" name = "id" /></p>
<p>Roll No. <input type = "text" name = "roll_no" /></p>
<p>Name <input type = "text" name = "name" /></p>
<p>Student class <input type = "text" name = "stud_class" /></p>
<p>Department <input type = "integer" name = "department" /></p>
<p><input type = "submit" value = "Submit" /></p>
</form>
```

3. Outputs/Results:

Django app

```
(hello_env) C:\Users\admin\scalable service\hellodjango>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
February 18, 2021 - 15:36:24
Django version 3.1.6, using settings 'hellodjangoproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Check the page in a browser



← → ↻ ⓘ 127.0.0.1:8000/studentlist/ 🔍 ☆ 🌐

📱 Apps 🕒 Birthday Invitation... 🖨️ 16 ch ken black sol... 🔄 Kubernetes exercises 🔄 django-channels2... 📄 Eduonix | Success 🔄 Worksheetsfun 🔄 BITS ERP SIS | Peopl... 🔄 The

Student ID	Roll No	Student Name	Class	Department
1	2019cg123001	Advait	Software Systems	CSE
2	2019cg123002	Asmi	Electrical Systems	ECE
3	2019cg123003	Anay	Database Systems	IT

Flask App

Open a new command prompt window, activate the environment variable then go to your “helloflaskapp” folder and run the following command to start the app

```
(hello_env) C:\Users\admin\scalable service\helloflask\helloflaskapp>python flasksharedb.py
* Serving Flask app "flasksharedb" (lazy loading)
* Environment: production
  WARNING: This is a development server. Do not use it in a production deployment.
  Use a production WSGI server instead.
* Debug mode: off
* Running on http://localhost:5000/ (Press CTRL+C to quit)
```

Now in the browser open <http://localhost:5000/shareddbform>

Enter some data



← → ↻ ⓘ localhost:5000/shareddbform

Apps Birthday Invitation... 16 ch ken black sol... Kubernetes ex

ID

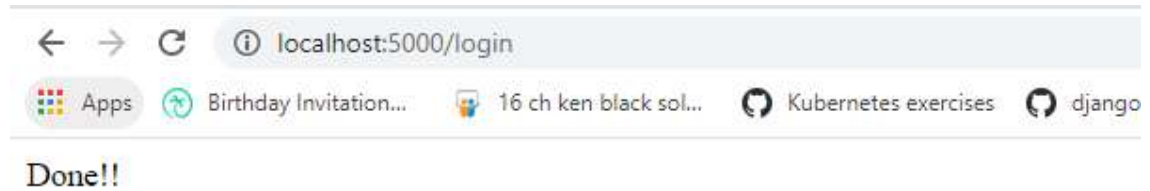
Roll No.

Name

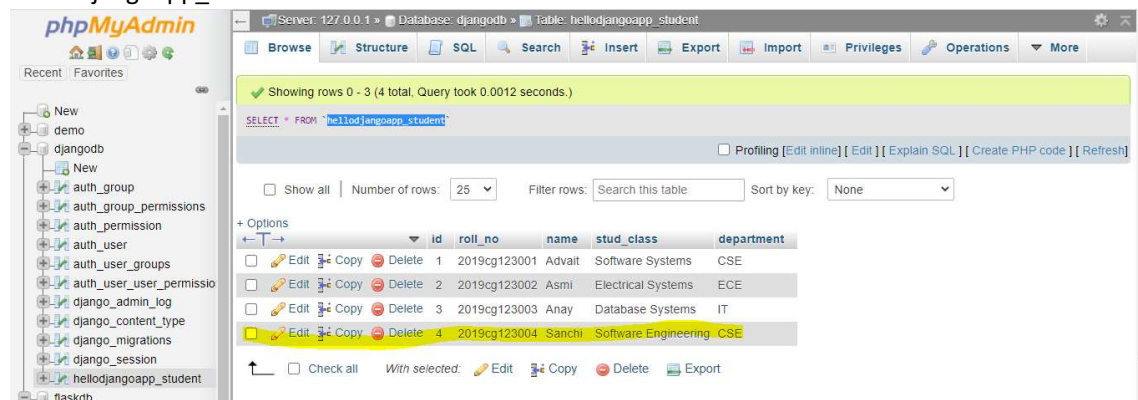
Student class

Department

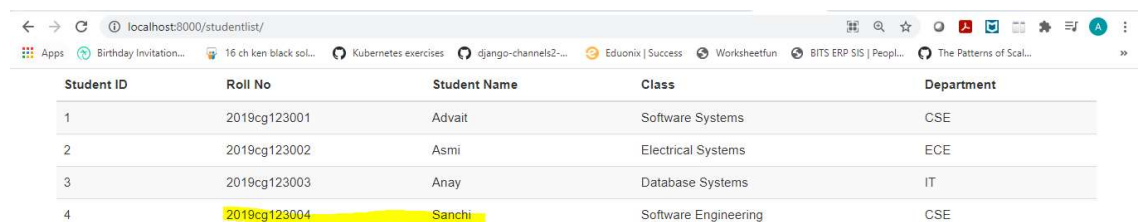
Now press submit



Now in the browser open <http://localhost/phpmyadmin>. In djangodb check the contents of “hellodjangoapp_student”



Now go to browser and open <http://localhost:8000/studentlist/> to check data available in Django app



Student ID	Roll No	Student Name	Class	Department
1	2019cg123001	Advait	Software Systems	CSE
2	2019cg123002	Asmi	Electrical Systems	ECE
3	2019cg123003	Anay	Database Systems	IT
4	2019cg123004	Sanchi	Software Engineering	CSE

4. Observations:

We are now able to communicate between the services by using shared database approach. Both the services are referring to same database, one service writes the data while another service reads the data.



5. References:

<https://docs.djangoproject.com/en/3.2/>

<https://flask.palletsprojects.com/en/2.0.x/quickstart/>